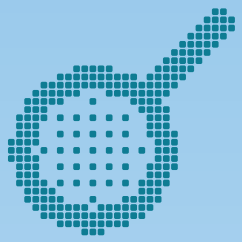

QSOE

Quick and Secure Operating Environment

Design & Architecture

One userspace, two interchangeable microkernels

Document:	`QSOE-DSN`
Revision:	Rev B — Draft
Applies to:	QSOE 0.2

Yuri Zaporozhets

August 2026

Contents

Chapter 1

Introduction

1.1 What is QSOE?

QSOE — the Quick and Secure Operating Environment — is a QNX-style operating environment for 64-bit RISC-V, built entirely under the Apache-2.0 license. It follows the QNX Neutrino tradition: a small kernel with everything else in userspace, synchronous message-passing IPC, and the resource-manager model for system services.

What makes QSOE unusual is that it is not one system but *two*, sharing a single userspace. The same QNX-style userland runs, byte-for-byte, on either of two interchangeable microkernels:

- **QSOE/L** — on **seL4**, the formally verified capability microkernel; and
- **QSOE/N** — on **Skimmer**, a from-scratch microkernel written for this project, derived in idiom from DragonFly BSD's light-weight kernel threads (LWKT).

The target platform is RISC-V rv64 (Sv39). Development is on the QEMU `virt` machine; QSOE also boots on real hardware — the SiFive HiFive Unmatched (FU740), and the SpacemiT K3 Pico-ITX, a sixteen-hart SoC that QSOE/N reached in 0.2. The K3 is QSOE/N only: it is too new for seL4.

1.2 The core bet: one userspace, two kernels

The central design bet of QSOE is that a complete QNX-style userspace can be made *100 % identical* across two very different kernels, with only a thin, well-defined seam differing between them.

Concretely, the per-kernel surface is exactly three things:

1. **The kernel** — Skimmer (QSOE/N) or seL4 (QSOE/L).
2. **taskman** — the central system server, the analogue of QNX's `procnto`. It sits directly on its kernel (seL4 endpoints and capabilities, or Skimmer channels and messages) and is therefore genuinely different per system, even though both link a large shared body of OS-independent logic.
3. **The OS-dependent part of the C library** — the per-kernel syscall/IPC seam beneath an otherwise shared `libc`.

Everything else — the shell, drivers, utilities, and the entire OS-independent body of the C library — lives once, in the shared `quser` tree, and is compiled identically for both systems. A user program built for QSOE is a single dynamically-linked binary that runs on either variant; only the system's `libc.so` differs at load time (Chapter 9).

This discipline is what the rest of this manual is really about: keeping the two systems converged at the userspace level while letting each kernel be exactly what it wants to be underneath.

1.3 Why two kernels?

The two kernels answer different questions, and QSOE deliberately refuses to choose between them.

seL4 brings a machine-checked proof of functional correctness and a capability model with the strongest isolation guarantees available in any production microkernel. Where the priority is assurance — a minimal, verified trusted computing base — QSOE/L is the answer.

Skimmer brings full control of the kernel itself: a from-scratch, preemptive, priority-driven, per-CPU design aimed at hard real-time, where the scheduler, the interrupt path, and the IPC fast path are ours to shape. Where the priority is determinism and direct hardware mastery, QSOE/N is the answer.

Because the userspace is shared, a program, a driver, or a resource manager written once is exercised on both — which keeps the QNX-style API honest and surfaces, at every milestone, any place where the two seams have quietly drifted apart.

1.4 What QSOE deliberately does not implement

A few classic Unix facilities are intentionally absent. These are design decisions, not gaps to be filled later:

- **No `fork()`.** Process creation is via `posix_spawn()` only. Without `fork()` there is also no payoff for copy-on-write, which is likewise a hard non-goal: each spawn maps fresh frames, and shared memory is explicit.
- **No `select()`.** `poll()` is the single multiplexer.
- **No `brk()`.** Low-level memory comes only through mmap-based calls.

These choices follow QNX practice and keep both kernels' memory and process models simple.

1.5 Heritage and licensing

QSOE is the unencumbered continuation of QRV, a finished port of QNX to RISC-V. QRV proved the hardware bring-up and the API shape; QSOE shares none of its proprietary code and is free to evolve. The whole of QSOE is Apache-2.0. Skimmer was modeled in structure and idiom on DragonFly BSD's LWKT and message-port subsystems, then written fresh. Imported files preserve their original license headers.

The name was coined as a play on *QSO* — the ham-radio term for a two-way contact — between seL4 and QNX, plus *E* for "established" (as a QSO is) or, just as fittingly, for "Environment".

1.6 Scope of this manual

This manual describes the *architecture* of QSOE: the structure shared by both variants and the precise points at which they differ. It is not a tutorial (see the User Guide, `QSOE-USR`) nor a programming reference (the Programming Book, `QSOE-PRG`; the App Porting Guide, `QSOE-APG`; and the C Library Reference, `QSOE-LCR`). The distributed-networking design has a manual of its own, the Networking Guide (`QSOE-NET`).

The remaining chapters proceed from the outside in:

- Chapter 2 — the system architecture: the software layers, the OS seam, and exactly what differs between QSOE/N and QSOE/L.

- Chapter 3 — the two kernels, side by side, and how each realizes the QNX IPC model.
- Chapter 4 — taskman, the central system server.
- Chapter 5 — channels, connections, messages, and pulses.
- Chapter 6 — the resource-manager model.
- Chapter 7 — identity and access: how a server learns who its client is, and what it does about it.
- Chapter 8 — drivers and the device model.
- Chapter 9 — the C library and dynamic linking.
- Chapter 10 — the boot path of each variant.

Chapter 2

System Architecture

This chapter looks at QSOE as a whole: the layers of software between the hardware and an application, which of those layers are shared between the two variants and which are not, and how the source tree is organized to keep that split honest.

2.1 The software stack

Figure 2.1 shows the stack. Read from the bottom up, it is the familiar microkernel shape: a small kernel in supervisor mode, and everything else — the system server, the C library, drivers, and applications — in user mode, talking by messages.

What is particular to QSOE is the dashed line. Everything *above* it is shared, compiled once and identical on both variants. Everything *below* it is the per-kernel seam. The whole engineering discipline of the project is to keep that line as low and as thin as possible.

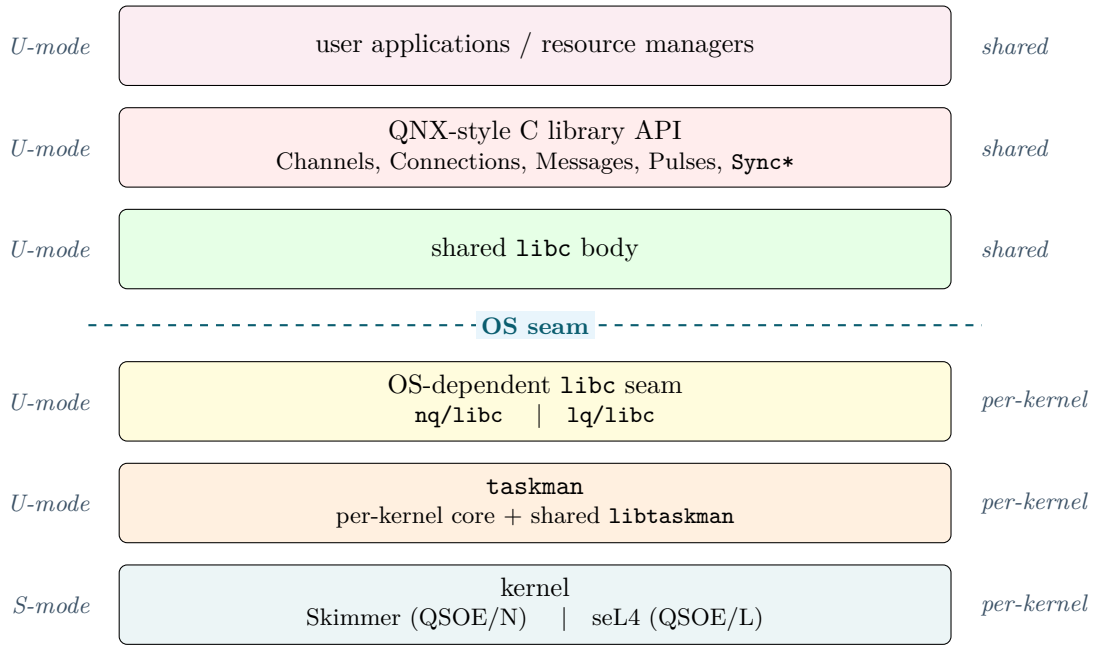

Figure 2.1: The QSOE software stack. Above the OS seam is shared and identical on both variants; below it is the thin per-kernel surface.

2.2 The OS seam: exactly three things

The per-kernel surface — everything below the dashed line — is exactly three things. Naming them precisely matters, because the promise of QSOE is that *nothing else* differs.

1. The kernel. Skimmer for QSOE/N, seL4 for QSOE/L. The two are not variations on a theme: Skimmer is a preemptive, priority-driven, per-CPU microkernel written for this project; seL4 is the formally verified capability kernel. They are described side by side in Chapter 3.

2. taskman. The central system server — QNX's `procnto` analogue — sits directly on its kernel's primitives and is therefore genuinely different per variant. But it is not written twice from scratch: the OS-independent body of every taskman (path management, credentials, system configuration, address-keyed synchronization, relocation) lives once in `libtaskman` and is linked into both. Only the core that touches the kernel — endpoints and capabilities on seL4, channels and messages on Skimmer — is per-variant. taskman is the subject of Chapter 4.

3. The OS-dependent part of `libc`. Beneath an otherwise shared C library, a thin seam implements the per-kernel syscall and IPC path: how a `MsgSend` actually reaches the kernel, how a pulse is delivered, how a thread traps. This seam lives in `nq/libc` and `lq/libc`, atop the shared `libc` body. The library as a whole is Chapter 9.

Everything not in that list — the QNX-style API surface, the entire OS-independent body of `libc`, and all of userland — is shared.

2.3 The shared userspace

The shared userspace lives in the `quser` tree: the shell, the utilities, the drivers, `sbin/init`, the resource-manager framework, and the OS-independent portion of the C library. It is compiled once and is *byte-for-byte* identical on both variants.

This is possible because user programs are **dynamically linked** by requirement, not by preference. Since the OS-dependent seam differs per kernel, a single portable binary cannot statically contain it; instead it binds the system's `libc.so` when it is loaded. A program built for QSOE is therefore one ELF that runs on either variant — it declares which libraries it needs, and each system supplies its own `libc.so`.

What binds them is `taskman`, not an interpreter inside the process. A QSOE binary names no loader at all: it carries no `PT_INTERP` header, and the central server walks its `DT_NEEDED` list, loads what it names, and applies every relocation before the program runs (Chapter 9).

2.4 Source-tree layout

The repository structure mirrors the architecture exactly: shared things live once, per-kernel things live under their variant.

Tree	Contents
`nq/`	QSOE/N: the Skimmer kernel and its taskman + `libc` seam
`lq/`	QSOE/L: the seL4 integration and its taskman + `libc` seam
`quser/`	the shared userspace — everything OS-independent
`libc/`	the shared C library tree (headers, body, `crt0`)
`libtaskman/`	the OS-independent body of every taskman

Table 2.1: Top-level source trees.

Two trees are worth a second look, because they are what make the shared/per-kernel split work in practice:

- `libc/` holds the C library *body* — the OS-independent code, the headers, `crt0`, and the run-time loader. Each variant's `nq/libc` and `lq/libc` sources the shared body underneath their own seam, so the bulk of `libc` is written once.
- `libtaskman/` holds the OS-independent body of taskman, linked into both `nq/taskman` and `lq/taskman` (§2.2).

2.5 Selecting a variant

Each component owns its own Makefile; the top-level Makefile only descends into them. Building QSOE/N is a build under `nq/`; building QSOE/L is a build under `lq/`. Both produce the same shared `modpkg.cpio` userspace package via the shared `quser` recipe, and each adds only its own kernel-side artifacts on top.

Within a variant, board selection is a build-time choice: a Kconfig menu of target boards — the QEMU `virt` machine, the SiFive Unmatched, the SpacemiT K3 — several of which may be selected at once. Both variants work this way.

That it is a build-time choice *at all* deserves a sentence, because the reason is narrower than it looks. Both kernels are FDT-driven and discover the machine at run time (§10.1), so a board is not a compiled-in personality: once a kernel derives its own physical load address at run time, no compiled object carries a board constant, and every board shares one set of objects. What remains per-board is the *link* step alone. The boot path for each variant is the subject of Chapter 10.

2.6 What is shared, and what diverges

Table 2.2 is the whole architecture in one view: for each part of the system, whether it is shared or per-kernel. It is, in effect, the project's contract with itself.

Component	**Shared or per-kernel?**
User applications, shell, utilities, drivers	shared
Resource-manager framework	shared
QNX-style C library API	shared
`libc` body, `crt0`	shared
`libtaskman` (taskman body)	shared
OS-dependent `libc` seam	per-kernel
`taskman` core	per-kernel
Kernel (Skimmer / seL4)	per-kernel

Table 2.2: The shared/per-kernel split, component by component.

The chapters that follow descend through the per-kernel rows of this table — the two kernels (Chapter 3) and taskman (Chapter 4) — and then back up through the shared rows: IPC (Chapter 5), the resource-manager model (Chapter 6), and the C library (Chapter 9). Two chapters cut across that order rather than sitting in it, because their subjects do: identity and access (Chapter 7) is carried by the seam, checked by the framework and owned by taskman, and the device model (Chapter 8) is what the shared rows exist to support.

Chapter 3

The Two Kernels

This is the one chapter that looks below the OS seam at the kernels themselves. Both present the same QNX-style message-passing model to the layers above; they realize it in very different ways. Skimmer is ours, written from scratch with a scheduler we control; seL4 is a formally verified capability kernel we build upon. The point of this chapter is to show that the *difference between them does not reach userland* — and exactly why.

3.1 Skimmer (QSOE/N)

Skimmer is a QNX-inspired microkernel for 64-bit RISC-V, skimmed in idiom from DragonFly BSD's light-weight kernel threads (LWKT) and message-port subsystems, then grown — in layers — into a QNX-style API surface (`MsgSend`/`MsgReceive`/`MsgReply`, channels and connections, interrupt attachment). The DragonFly files were studied as a structural reference and then discarded; none of the current source is a derived work. The file layout and idiom echo DragonFly, but the code is ours and freshly written under Apache-2.0.

Being our own kernel, Skimmer is shaped by four design commitments:

1. **Per-CPU first; cross-CPU by message.** Per-hart state needs no locks; spinlocks exist only for genuinely cross-CPU paths. Pushing more state into a per-hart structure is cheaper than locking it.

2. **Preemptive, priority-driven scheduling.** The LWKT *mechanisms* are kept (per-CPU run queue, IPI-based migration); the LWKT *policy* (cooperative within a priority band) is replaced with strict priority preemption, so an interrupt service thread can displace a lower-priority thread on any hart immediately. This is what makes hard real-time possible (up to N hard-real-time tasks on an N-CPU system).

3. **Msgports are the transport, not the API.** The LWKT message ports are kernel-internal plumbing; the user-facing QNX `MsgSend` — rendezvous-only, with defined copy-/IOV semantics and priority inheritance — is built *on top of* them, not aliased to them.

4. **Personas are conceptual, not structural.** A user thread that traps into the kernel is the same thread: same TID, same priority, same run queue. Only the privilege level, address space, and stack pointer change. "User persona" and "kernel persona" never become two scheduler entities.

Userland sees Skimmer through a small, deliberately grouped syscall ABI (channels, messages, interrupts, timers, clocks, synchronization, threads). Because there is no QNX RISC-V ABI to preserve, the message layout, syscall numbers, and pulse format are free design choices. In keeping with a microkernel, the syscall set does not grow casually: new system functionality is added by sending messages to taskman, not by adding syscalls.

Skimmer has no demand paging and no copy-on-write (§1.4), and supports SMP from the start.

3.2 seL4 (QSOE/L)

seL4 is the formally verified capability microkernel. QSOE/L does not modify it; it builds the QNX programming model on top of seL4's endpoints, notifications, and capability-managed address spaces. Only the seL4 kernel runs in supervisor mode; everything else — taskman included — is a user-mode process, giving QSOE/L a minimal, proven trusted computing base.

Where Skimmer hands us the scheduler, seL4 hands us a capability system: each process has its own CSpace and VSpace, every spawn building fresh page tables, and authority is conveyed only by capabilities. QSOE/L runs on the MCS configuration of seL4, which provides the scheduling contexts and native timeouts the QNX timer and synchronization surface needs.

3.3 Mechanism, not policy

The reason one userspace can sit on two such different kernels is a strict separation of *mechanism* from *policy*.

Each kernel provides the low-level *mechanism*: a way to rendezvous two threads and copy a message, a way to signal asynchronously, a handle that names a destination. Neither kernel decides the *policy*: who owns a channel, who allocates a connection, who revokes it on exit, how a name resolves to a server. That policy lives entirely above the seam — in taskman (Chapter 4) and the per-process C library (Chapter 9).

This is why the kernel mapping below describes only mechanism. Two kernels can realize the same QNX concept with different primitives precisely because the policy that gives those primitives their QNX meaning is written once, above them both.

3.4 Mapping QNX concepts to each kernel

Table 3.1 gives the correspondence: for each QNX concept, the mechanism that realizes it on each kernel. It is the concrete form of §3.3.

QNX concept	**Skimmer (QSOE/N)**	**seL4 (QSOE/L)**
Channel	Kernel channel object	Endpoint
Connection (`ConnectAttach`)	Connection to a channel, named by a coid	Capability to an Endpoint
`MsgSend` / `MsgReceive` / `MsgReply`	Rendezvous message syscalls, carried over LWKT msgports	Synchronous IPC (`seL4_Call` / `seL4_Recv` / `seL4_Reply`)
Pulse (`MsgSendPulse`)	Asynchronous pulse message	Notification
Connection ID (coid)	Process-local connection handle	Capability badge
Receive ID (rcvid)	Receive handle for the pending reply	Badge value on the incoming message
Channel ownership	Held by taskman	Master Endpoint capability held by taskman
Resource manager	User-space server	User-space server holding capabilities

Table 3.1: QNX concepts and the mechanism that realizes each, per kernel.

The table describes the *mechanism* only. It does not say who holds which handle, who allocates, or who revokes — that separation is deliberate, and the policy is the subject of the chapters above the seam.

One difference is worth naming, because it is visible to an architect even if not to an application: how a channel is represented. On Skimmer a channel is a global kernel object identified by a channel id; on seL4 it is a per-process Endpoint reached through a capability. The QNX programming model is faithfully preserved on both — code to the per-process model, discover servers by name, and request a global channel only when you explicitly mean to — so the distinction stays an implementation detail beneath the C library.

3.5 Why the difference does not reach userland

Above the seam, none of this is visible. taskman wraps the kernel mechanism into the QNX policy; the C library seam wraps the syscall path into the QNX API. An application calls `MsgSend` and receives a reply with identical semantics on either kernel — the rendezvous, the copy semantics, the priority inheritance are all specified by QSOE, not by the kernel underneath.

The remaining chapters live above the seam and are therefore written once for both variants: taskman (Chapter 4), the IPC model in full (Chapter 5), the resource-manager framework (Chapter 6), and the C library that presents it all (Chapter 9).

Chapter 4

taskman — The Central System Server

If the kernel is the mechanism, `taskman` is where the policy begins. It is the first user process the system starts, and every other process descends from it. This chapter describes what taskman is responsible for, how it is split between a shared body and a per-kernel core, and the main service domains it provides.

4.1 Role

`taskman` is QSOE's root server, and the direct analogue of QNX's `procnto`: a single userspace process that combines the responsibilities of a process manager, a memory manager, a path (namespace) manager, and a provider of system services.

The crucial difference from QNX is structural. In QNX, `procnto` is fused with the microkernel into one privileged image. In QSOE, the kernel provides only the bare mechanism (Chapter 3), and `taskman` is an ordinary user-mode process that implements all higher-level OS policy from outside the kernel. It is powerful by virtue of the capabilities and registries it holds, not by virtue of privilege: on QSOE/L it runs in U-mode like any other process; on QSOE/N likewise.

4.2 The shared body and the per-kernel core

taskman is the most variant-specific component in userland — it sits directly on its kernel's primitives — yet it is not written twice. Its OS-independent logic lives once in `libtaskman` and is linked into both `nq/taskman` and `lq/taskman`:

- **pathmgr** — the pathname space: registration, resolution, mounts, and symlink expansion (§4.7).
- **cred** — the credential model and the privilege policy (§4.9).
- **syscfg** — system configuration and the `/sys` model (§4.8).
- **sync** — the address-keyed wait/wake that backs the QNX `Sync*` API.
- **reloc** — relocation support for loaded images.

Only the *core* that touches the kernel is per-variant: endpoints, capabilities, and untyped memory on seL4; channels, messages, and frames on Skimmer.

A bootstrap paradox (QSOE/L only). This is a place where the mechanism-vs-policy split of Chapter 3 becomes concrete. On QSOE/L a channel is an seL4 Endpoint that taskman manages by *policy*, so `ChannelCreate` is implemented as a message *to* taskman — which creates a chicken-and-egg problem at startup: taskman cannot use `ChannelCreate` to make its own first channel, because no channel exists yet to carry the request. It therefore creates its primary Endpoint by hand, directly on seL4, and registers itself as the first entry (`pid=1`) in the channel registry; from then on every channel is created through the normal API path. On QSOE/N the question never arises: a channel is a Skimmer kernel *mechanism*, so `ChannelCreate` is an ordinary system call and taskman simply calls it like any other process.

4.3 Service domains

taskman multiplexes several service domains over its primary channel, dispatching on the message type (Table 4.1). Each corresponds to a responsibility QNX folds into `procnto`.

Domain	Operations	QNX equivalent
Process	spawn, terminate, wait/reap	`procnto` process manager
Memory	`mmap`, `munmap`, shared memory	`procnto` memory manager
Pathname	registration, resolution, `open` routing	`procnto` path manager
Session	sessions, process groups, controlling terminal	`procnto` session manager
Identity	credentials, authentication (Ch. 7)	`procnto` + setuid binaries
System	system info, configuration, shutdown	various `procnto` calls

Table 4.1: taskman service domains.

All interaction with taskman follows the QNX Send/Receive/Reply pattern: a client packs a request, sends it to taskman's channel, and blocks for the reply. taskman runs a receive–dispatch–reply loop, handling one request and immediately waiting for the next. The IPC model itself is the subject of Chapter 5.

4.4 Process management

QSOE has no `fork()` (§1.4); process creation is via `posix_spawn()` only. A spawn is a single transaction with taskman: the client supplies the executable path and the new process's arguments and environment, the latter through a small side-channel page rather than inline in the message. taskman loads the image, builds a fresh address space, sets up the new process's first thread, and returns its pid.

Because there is no `fork()`, there is also no copy-on-write: each spawn maps *fresh* frames (§1.4). How those frames are obtained is the per-kernel part: on QSOE/L taskman retypes untyped memory into a new CSpace and VSpace under a per-process budget; on QSOE/N it allocates and maps fresh frames directly. The result is the same — a new, isolated process — reached two different ways below the seam.

When a process exits, taskman reaps it: it reclaims the process's frames, capabilities, and registry entries, retires its channels and connections, and makes its exit status available to `waitpid()`. A faulting process is torn down the same way, with its peers notified so a blocked `MsgSend` to it fails cleanly rather than hanging.

4.5 Sessions and the controlling terminal

A machine with two consoles needs "my terminal" to mean something per-process, so taskman keeps a *session*: a refcounted object holding a session leader, a process group, and a controlling

terminal, shared by every process spawned within it.

The refcount is why it is a separate object rather than fields on the process. Setting the terminal once — in `getty`, before it spawns `login` — is then seen by `login`, by the shell that `login` spawns, and by everything the shell spawns after that. That inheritance is the entire point; per-process copies would have to be re-established by every child.

This is also why `/dev/tty` cannot be a symbolic link. It names "the controlling terminal of whoever is asking," which is a *per-caller* answer that no global registration can express. On a machine with a serial console and a screen, a single alias necessarily names the wrong one for half the system — concretely, a `login` running on the screen printed its password prompt down the serial port and waited for keystrokes there. taskman therefore answers `/dev/tty` itself, resolving it from the asking process's session, exactly as `procnto` does.

QSOE stores the terminal as a *path* where QNX stores a descriptor procnto holds open. The reason is local: a QSOE descriptor is a raw connection id onto a resource server and cannot be mapped back to a name, while the path manager resolves by path and rewrites the path in its reply — which is precisely the channel `/dev/tty` needs, so that the terminal's server learns which device was opened.

4.6 Memory management

Low-level memory is obtained only through mmap-based calls; there is no `brk()` (§1.4). `mmap` and `munmap` route through taskman, which owns the per-process address space. The allocator is a bump cursor over the virtual address range: `mmap` advances the cursor and backs the new range with fresh frames; `munmap` releases the frames but does not rewind the cursor. Shared memory is always explicit — there is no implicit sharing to inherit, precisely because there is no `fork()`.

4.7 Path and namespace management

taskman (through `libtaskman`'s pathmgr) owns the system's single pathname space. Servers register the path prefixes they are responsible for; when a client opens a path, taskman resolves it — following mounts and symlinks — to the server that should handle it, and the client is connected to that server. This is the mechanism by which a program reaches a resource manager *by name* without knowing where it lives; the resource-manager framework that sits on the other end is Chapter 6.

4.8 System configuration: the `/sys` model

Rather than answer a dedicated configuration IPC for every query, taskman exposes system facts through a small read-only model surfaced under `/sys` (for example `/sys/osname`, `/sys/version`, `/sys/board`, the system's hostname, and the table of current mounts). The model itself lives once in `libtaskman`; each variant feeds it from its own source of truth — the kernel-built information page on QSOE/N, the equivalent configuration path on QSOE/L — so the same `/sys` entries appear on both systems. A `uname`-style tool, or `getty`'s banner, reads these the same way regardless of kernel.

4.9 Credentials

taskman holds the credential model (user and group identity) and enforces the privilege policy in one place, in `libtaskman`'s cred module. The policy is the conventional one: the superuser (effective uid 0) is unrestricted, while an unprivileged process may only assume identities it

already holds — its own real, effective, and saved ids — so it can shuffle among them but can never gain a new one, and in particular cannot `setuid(0)`. Because the rule lives in the shared body, both variants' taskman enforce it identically.

taskman also owns the system's password database, which is what allows a process to acquire an identity it does *not* already hold without any program in the system carrying a setuid bit. That, and the mechanism by which a server learns which client it is serving, are the subject of Chapter 7.

Chapter 5

IPC: Channels, Connections, Messages, and Pulses

Message passing is the heart of a QNX-style system, and of QSOE. Almost everything a program does that is not pure computation — opening a file, reading a device, asking taskman to spawn a process — is a message to a server. This chapter describes the model: how endpoints are named, how a synchronous message exchange works, and how asynchronous pulses complement it. The model is identical on both variants; the kernel mechanisms that realize it were given in Chapter 3.

5.1 Channels and connections

Communication is between a *channel* (the server side) and a *connection* (the client side).

A server creates a channel with `ChannelCreate` and then waits on it to receive messages. A client reaches that server by establishing a connection to the channel with `ConnectAttach`, which yields a *connection ID* (coid) — a small integer the client uses thereafter to send. The asymmetry is deliberate: one channel, many connections; the server receives, the clients send.

A client rarely calls `ConnectAttach` with raw coordinates. Instead it opens a *name*: taskman's path manager (§4.7) resolves the name to the owning server and arranges the connection. This is how a program reaches `/dev/console` or a filesystem without knowing which process serves it.

5.2 The Send/Receive/Reply rendezvous

The core exchange is synchronous and unbuffered — a rendezvous between two threads:

1. The client calls `MsgSend` on a coid and **blocks**.
2. The server's `MsgReceive` returns the message together with a *receive ID* (rcvid) naming this particular pending exchange.
3. The server processes the request and calls `MsgReply` with a status and an optional reply payload.
4. `MsgReply` unblocks the client; `MsgSend` returns the status the server passed.

There is no message queue and no buffering in the middle: if no server thread is receiving, the sender simply waits, and vice versa. A QNX implementation classifies a thread's blocked state precisely by which half of the rendezvous it is waiting on — SEND-blocked (the server has

not yet received), REPLY-blocked (received, not yet replied), or RECEIVE-blocked (a server idle with no client) — and QSOE preserves those semantics.

Because the exchange is synchronous, it carries priority with it: while a server handles a client's message it runs at the client's priority, so a high-priority client is not held off behind lower-priority work in the server. This priority inheritance is part of the contract, specified by QSOE rather than inherited from the kernel underneath.

5.3 coids and rcvids

The two handles name the two ends of an in-flight exchange. A **coid** is the client's process-local handle for a connection; a **rcvid** is the server's handle for a received-but-not-yet-replied message, the token it must present to `MsgReply`.

Following QNX faithfully, connection IDs and file descriptors share *one* integer namespace: a coid *is* an fd and an fd *is* a coid. An `open` returns a descriptor that is also a connection one can `MsgSend` on directly, and there is no separate descriptor table to keep in step — which is what lets the same `read`/`write`/`close` calls work uniformly on files, devices, and raw connections. Also as in QNX, a reserved *side-channel* range is set aside for the library's own internal connections (for instance the standing connection to taskman) so they never collide with application descriptors.

5.4 Message and reply payloads

A message is a block of bytes whose meaning is a contract between client and server; the first word is conventionally a message type that the server dispatches on. Both the send and the reply may be gathered from, or scattered into, several buffers (an I/O vector), so a header and a body in separate buffers need no prior concatenation.

The reply is *pure payload*. The server's completion status is not smuggled into the reply bytes: `MsgReply` takes the status as a separate argument, and the client receives it as the return value of `MsgSend`. Keeping status and payload distinct means a server can report an error with no reply body, and a client always knows the outcome without parsing the data.

5.5 Pulses

Not every notification wants a reply. A *pulse* is a small, fixed-size, asynchronous message: the sender does not block and there is no reply. Pulses carry a code and a value, and are the right tool for events and readiness notifications — a driver telling a server that data has arrived, a timer firing.

Pulses also carry **signals**. QSOE delivers POSIX signals as pulses to a per-process *system thread*: every program runs, from the first instant, two threads — its `main` thread and a system thread that receives these pulses and runs signal handlers. A consequence worth stating plainly is that Skimmer has *no* `Signal*` system call: signal delivery is pulse delivery, with no separate kernel mechanism. The same model holds on QSOE/L.

5.6 Bulk transfers

The rendezvous is tuned for the common case of small messages, but some exchanges are large — reading a megabyte from a file, say. For these, QSOE carries the payload in bulk, copying it page by page directly between the two address spaces rather than forcing the application to set up shared memory by hand. A generous upper bound keeps a single transfer from being unbounded. The application sees an ordinary large `MsgSend`; the page-by-page movement is below the API.

5.7 Distributed transparency

Several IPC entry points carry a *node descriptor* argument naming the station a connection reaches. Through 0.1 it could only name the local node, and the argument was a deliberate placeholder. Since 0.2 it is live: a Send/Receive/Reply exchange can cross a wire.

What that buys is stated most clearly by what a program does *not* do. `open("/net/k3/dev/ser1")` opens the serial port on the station called `k3`, and neither end is told that anything unusual happened: the client issues an ordinary `open` and an ordinary `read`, and the resource server on the far station sees an ordinary local client. Every driver and every resource server already written therefore works across the net unmodified. That is why this belongs in an architecture manual rather than in a list of features — it is a property of the message-passing model, not a service added beside it.

Two choices make it hold. The relay deliberately *does not decode*: exactly two message types are inspected — the open that names the resource and the close that releases it — and everything else crosses as opaque bytes, so a client and server with their own private protocol pass through unchanged. And station ids are globally unique rather than node-local, so a descriptor that crosses a boundary needs no translation and `struct _msg_info` keeps a single node field.

The protocol is **QSP**, which rides directly on Ethernet with no IP. Its frame format, the roster and call-sign model, and an honest inventory of what 0.2's transport does and does not yet guarantee are the subject of the Networking Guide (`QSOE-NET`). The architectural fact recorded here is only this: the node descriptor is no longer a placeholder.

Chapter 6

The Resource-Manager Model

In a microkernel, a file is not a kernel object — it is a conversation with a server. So are a device, a directory, and a pseudo-terminal. The *resource-manager model* is the pattern those servers follow, and it is how QSOE presents a POSIX face (`open`, `read`, `write`, `stat`, ...) over message passing. This chapter describes the model and the framework QSOE provides for writing it.

6.1 What a resource server is

A *resource server* is an ordinary process that registers one or more paths in the system namespace and answers client I/O on them. A word on terms, used consistently throughout: following QNX, the *pattern* is the *resource-manager model*, while a concrete process that implements it is a *resource server* — the model is a manager, a thing built to it is a server. (taskman's *path manager* of §4.7, and its process and memory managers, are unrelated components that merely share the word.)

A resource server ties together the two preceding chapters: it registers its paths with taskman's path manager (§4.7), and it serves clients over the Send/Receive/Reply IPC of Chapter 5.

The client never sees any of this. When a program calls `open("/dev/console", ...)`, the path manager resolves the name to the console server and connects the client to it; the returned descriptor is a connection (§5.3), and the program's subsequent `read` and `write` are messages to that server. "Everything is a file" becomes "everything is a server you reach by name."

6.2 A clean-room framework

QNX programmers will recognize the model; QSOE does *not* reuse QNX's implementation of it. The QNX `resmgr`/`iofunc` layer is proprietary and is never ported. QSOE instead provides its own framework, designed from scratch, that keeps the familiar shape while making two deliberate choices of its own:

1. **A server is expressed as objects.** Each resource is one C structure — a *Provider* — carrying both the resource's data and a pointer to its method table. The model is defined so that one Provider projects cleanly onto a single object: its method table is that object's virtual-method table. A server author writes objects, not message handlers.

2. **The framework owns the loop.** The receive–dispatch–reply loop belongs to the framework. It decodes each incoming request into *typed* arguments and invokes the one method that names the operation; a method never sees an undifferentiated message buffer. There is no reachable raw buffer to mis-parse or overrun.

6.3 Providers, handles, and methods

Three concepts carry the model.

Provider. The resource object. It holds the resource's POSIX attributes (its "inode": mode, owner, size, timestamps), its registered path, and — by subtyping, embedding the base as its first member — whatever private state the resource needs. Its first member is the pointer to its method table, shared by all instances of the same type.

Handle. One open instance, created by the framework when an open is accepted and destroyed after the matching close. It carries the per-open state: the current offset, the accepted flags, and an author pointer for anything else. One Handle per successful `open`.

Methods. The Provider's method table groups the operations a resource can answer. Each takes the Provider as an explicit *Self* argument, so the same table serves every instance:

- **lifecycle** — `acquire`/`release`: admit or refuse an open, and drop its state on close;
- **leaf I/O** — `pull`/`push`/`seek`: read, write, reposition;
- **attributes** — `query`/`adjust`: `stat`, and chmod/chown/truncate/utime;
- **control and readiness** — `control` (the typed, bounded escape hatch in place of QNX's `devctl`) and `ready` (which `poll` events are satisfiable now);
- **asynchronous** — `service`/`cancel`: an interrupt or internal event, and a client abandoning a blocked request;
- **directory** — `lookup`/`list`/`make`/`unlink`, present only on Providers that manage a namespace subtree.

A method slot may be left empty: a `NULL` slot has a defined meaning, and the framework supplies *correct-by-default* base methods (permission checks in `acquire`, attribute copies in `query`, offset arithmetic in `seek`). A minimal server therefore overrides only `pull` and `push` and inherits the rest.

The default `acquire` is the system's one enforcement point for POSIX permissions: it evaluates the mode triads against the credentials the client's message carried, and refuses the open if the bits are not there. A server author gets that by writing nothing, which is the argument for correct-by-default methods in one line. How the client's identity reached the method, and what else is checked with it, is Chapter 7.

6.4 The reply contract: synchronous or deferred

A request-bearing method produces *exactly one* reply, in one of two ways. It may return **synchronously** — the count read, the new offset, or a negative `errno` — and the framework turns that into the reply at once. Or it may **defer**: return a sentinel saying "not yet," and complete the call later.

Deferral is what makes drivers possible. A serial read with no data waiting cannot answer immediately; the driver defers it, and when the receive interrupt arrives, its `service` method completes the parked call with the bytes. The in-flight call is named by an *opaque* token: the author can complete or cancel it but cannot reach the underlying message, so there is no raw buffer to corrupt — a deliberate improvement over the exposed context object of the QNX framework.

6.5 Servers stay above the kernel

One discipline keeps the model portable: a resource server *never* calls the kernel directly. It uses only POSIX and the QSOE IPC primitives, never seL4 or Skimmer system calls. This is what lets the very same server binary run on both variants — the resource servers live entirely above the OS seam (Chapter 2), in the shared userspace.

6.6 Examples and the FreePascal projection

The framework is the single way servers are built: the `/dev/console` server, the serial-port driver, the pipe server, the read-only filesystems that mount the boot archive and the on-disk `fs-qrv`, and `fs-tmpfs` — the first writable filespace in QSOE — are all Providers with method tables, differing only in which methods they implement. So is `qsp`'s `/net` (§5.7), which is why a remote path resolves through the same machinery as a local one.

The object orientation is not incidental. The model is specified so that one Provider maps to exactly one FreePascal `object` and its method table to that object's virtual-method table — so a resource server can be written naturally in FreePascal as well as C. The full data types, the per-method contract, and the request-message (`_IO_*`) layer beneath the framework are the subject of the Programming Book (`QSOE-PRG`); this chapter has described only their shape.

Chapter 7

Identity and Access

Every chapter so far has described how a request reaches a server. This one describes how the server knows *whose* request it is, and what it is entitled to do about it.

The subject spans three layers, which is why it is a chapter of its own rather than a section of any of them. Identity is carried by the kernel seam (Chapter 3), checked by the resource-manager framework (Chapter 6), and owned by taskman (Chapter 4). No one of those three can be understood without the other two.

7.1 The question a server must be able to ask

A resource server enforcing POSIX permissions needs one fact: the identity of the client whose message it is holding. In a monolithic kernel this is free — the file system code runs inside the kernel and reads the calling process's credential structure directly. In a message-passing system it is not free at all, and the way it is made available shapes everything else.

The obvious design is for the server to *ask*: receive the message, then send a query to taskman naming the client. QSOE does not do this, and the reason is a deadlock rather than a matter of taste. taskman's dispatch is a single thread. While it is parked in a `MsgSend` to a file system — as it is whenever it opens an executable in order to spawn — it cannot answer that file system's question about its client. A server that asks hangs; and because taskman opens files on a file system in order to start processes, what hangs is the first spawn after the root file system mounts.

So identity is **delivered**, not looked up. The sender's credentials arrive with the message, in `struct _msg_info` alongside the pid, the priority, and the connection ids, and a server reads them from a structure it already has. There is no second exchange and therefore nothing to deadlock.

7.2 Carrying identity across two kernels

This is a seam problem, and the two kernels solve it differently — which is exactly the situation Chapter 3 describes as normal: the same guarantee above the seam, two mechanisms below it.

QSOE/N. Skimmer stamps the sender's credentials into the message information at *send* time, reading them from the sending process's entry as taskman last set it. All six ids travel — real, effective and saved, user and group.

QSOE/L. seL4 delivers a message with no room for anything alongside it except the receiver's own *badge* on the endpoint capability the sender used. The badge is therefore where the credential rides. It is one 64-bit word already carrying the server-side connection id, so the layout is the design:

Bits	Field	Width
0–29	server-side connection id (scoid)	30 bits
32–47	effective uid	16 bits
48–63	effective gid	16 bits

Two properties of that table are load-bearing and neither is arbitrary.

First, the credential begins at bit 32 rather than immediately above the scoid, because the *entire low word* is territory a receive id is cut from: a rcvid carries its own flag bits up there, and a credential overlapping them corrupts the token a server must present to `MsgReply`. The gap between bit 30 and bit 32 is not waste; it is the boundary.

Second, the layout is **fail-safe**. A badge that is lost, truncated or never stamped reads as zeroes, and zero is uid 0 — the superuser. A layout that fails *open* in a permission system is not a bug waiting to happen, it is the bug. QSOE/L therefore refuses to mint a badge for an id it cannot represent rather than truncating one: a truncated `0x10000` would land on zero, which is root. The 16-bit width is the price of that refusal, and on a system with a few dozen accounts it is not a price at all.

Only the effective pair fits. Where QSOE/N reports all six ids, QSOE/L reports the other four *equal to* the effective pair, so that no macro reading `struct _msg_info` can find a zero and mistake it for root. A server that genuinely needs the real and saved ids apart calls `ConnectClientInfo`, which asks taskman — outside the handling of a request, where asking is safe.

7.3 Current, not frozen

That credentials are read at *send* time, and not at connect time, is the second load-bearing half.

Consider the ordinary boot sequence: `getty` opens the console as root, prints its banner, and spawns `login`; `login` authenticates somebody and drops to their identity; the shell inherits it and runs for the rest of the session on the same descriptor. A credential captured when that descriptor was opened would report *root* for every keystroke typed afterwards, for as long as the machine stayed up.

Reading at send time reports the sender's identity *now*. A connection opened before a change of identity does not go on claiming the identity that opened it.

POSIX's own rule — that an open file's access rights are fixed at `open` — is not contradicted by this; it is *implemented on top of it*. A server wanting that freeze copies the credentials into its per-open state when it accepts the open, which is where a freeze belongs. The transport reports the truth, and a server decides what to remember. The converse arrangement, a transport that freezes on the server's behalf, offers no way back: a frozen credential cannot be un-frozen by anyone.

7.4 Checking it: the permission triad

Delivering identity is worth nothing until something acts on it, and through 0.1 nothing did — not as a policy, but because the framework's default `acquire` admitted every open it was given. Mode bits existed and were inert. `/etc/shadow` could be mode 0600 and readable by anybody, and no amount of adjusting permissions would have changed that, because permissions were not the thing being consulted.

The framework's default `acquire` now performs the conventional POSIX triad check (§6.3): from the requested access mode it derives the rights needed, selects the owner, group or other triad by comparing the client's ids against the object's, and refuses with `EACCES` if the bits are not there. uid 0 bypasses the triads, as it does everywhere.

Because it is the *default*, every Provider that does not override `acquire` gets the check without its author writing anything — which is the point of correct-by-default methods. A Provider that does override it (because admission depends on more than mode bits) calls the same check explicitly rather than reimplementing it, so there is one triad evaluation in the system and not one per server.

7.5 Describing is not accessing

`stat` does not fit that rule, and the way it does not is instructive.

POSIX requires no permission on a file in order to `stat` it — only search permission on the directories leading to it, which the path resolver has already applied by the time any server is reached. A `stat` implemented as an open followed by a query would therefore fail on precisely the files the triad check exists to protect: `ls -l` in a directory containing `/etc/shadow` would report an error rather than a line.

So a path-form `stat` is not an open. It is a request in its own right, which taskman resolves and forwards to the owning server, and the framework serves it by creating a temporary handle marked *metadata only*. The default `acquire` sees that mark and returns before the triad check, because describing an object is not accessing it. The mark is set by the framework alone and never travels on the wire, which matters: a client able to set it could turn every open into an unchecked one.

7.6 Authentication as a message

The remaining question is how a process legitimately *becomes* somebody else.

On Unix the answer is the setuid bit, and the reason is a genuine gap in that architecture: the credential authority is the kernel, while the password database is userspace policy the kernel knows nothing about, so a privileged program is needed to stand in both worlds. `su` and `login` are setuid-root because there is nowhere else for them to be.

QSOE has no such gap. taskman already owns the credential table; given the system's password files it owns the database as well. There is nothing left to bridge — so there is no setuid bit anywhere in QSOE, and with it goes the entire class of attacks that a privileged binary plus a mandatory dynamic loader has historically produced.

taskman caches the three system files (`/etc/passwd`, `/etc/group`, `/etc/shadow`) through a seam that reads them from whatever file system holds them, and answers exactly one question about them: *here is a name and a password — may this caller become that user?* `su` is an ordinary unprivileged program that asks it.

Three properties of that verb are deliberate.

The hash never leaves. Verification happens inside taskman. A caller sends the password and is told whether it became the user, never what the stored hash is. `/etc/shadow` therefore need not be readable by anything at all, and the file system's own triad check on it (§7.4) becomes a second line of defence rather than the only one.

One verb, not two. There is deliberately no "check this password" that a caller may invoke without also switching. Split in two, the pair is a password oracle: anyone could probe credentials and read the answer off which call failed. Authenticating and becoming are one operation, so a wrong password and a right one differ only in whether the caller is now somebody else.

One failure, not two. An unknown user and a wrong password both return `EACCES`. The difference between them is exactly what an attacker is asking for.

The cache is loaded after the root file system mounts rather than at taskman's start, because that is where the files live — and it is reloaded by an explicit request, which must come from somebody other than the file system holding them, for the single-threaded reason of §7.1.

7.7 Rate-limiting a central verb

Centralizing authentication is what removes the setuid binary. It also creates one call that any process may make in a tight loop, and a design note that stopped there would have replaced a slow password oracle with a fast one. `login` has always had three tries and a pause; this verb had neither.

Two limits, because either alone is defeated:

- **Per caller.** After a small number of consecutive failures the caller is refused for a lockout interval, cleared by a success.
- **Globally.** No two attempts closer together than a minimum interval, whoever makes them.

The per-caller count is keyed on **uid**, not on pid, and that distinction is the whole of its usefulness. Every `su` is a fresh process: keyed on pid, each attempt arrives with a number never seen before, the count reads one every time, and the lockout never fires. A uid cannot be shed to escape the count either, because shedding it *is* the operation being attempted and it requires the password. Keying on uid also scopes the lockout to the attacker, so one user cannot lock another out by guessing at their account.

The global limit is a *rate* limit and never a lockout: it slows everyone equally and locks nobody out, so it cannot be turned into a way of denying an operator their own console.

Both refusals are immediate. Sleeping to enforce a delay would park taskman's single dispatch thread and stop the whole system — a far better attack than the one being prevented. The system says no at once and counts the attempt.

7.8 What is not yet enforced

Stated rather than implied away, in the manner of the rest of this manual.

- **Path traversal is not checked.** The triad is evaluated on the object a path resolves to, not on the directories traversed to reach it. A file inside a directory whose execute bit is clear is still reachable by naming it directly.
- **Not every server checks.** The default `acquire` checks; a Provider that overrides `acquire` for its own reasons may not, and `fs-tmpfs` currently does not.
- **Credentials do not yet cross the net.** A QSP circuit carries a uid and gid in its connect frame and the far station does not act on them (§5.7); a net is a single trust domain. Nor is the identity carried the originating client's — it is the relay's own, which is the first thing to correct when the far side begins to enforce.

None of these makes the mechanism this chapter describes optional. They are places where an enforcement point is missing, not places where identity is wrong.

Chapter 8

Drivers and the Device Model

A driver in QSOE is not a special kind of program. It is a resource server (Chapter 6) that happens to own a piece of hardware — an ordinary user-mode process, dynamically linked against the same `libc.so` as the shell, holding no capability the kernel grants it by class and running on either variant unchanged.

That is worth stating plainly because it is the single most consequential difference from a monolithic system, and everything else in this chapter follows from it. A driver fault does not take the kernel with it. A driver can be restarted. A driver can be debugged with the ordinary tools. And a driver written for one of QSOE's two kernels runs on the other, because it never sees either.

8.1 Naming and shape

QSOE keeps the QNX naming convention, which encodes the class of device in the program's name: `devc-` for character devices, `devb-` for block devices, `devn-` for network interfaces, `devu-` for USB hosts, and `fs-` for file systems. So `devc-ser8250` is a UART driver, `devb-nvme` an NVMe disk, `devn-gem` the Cadence Ethernet controller, `fs-tmpfs` a file system.

Each registers a path with taskman's path manager and answers I/O on it. A program that opens `/dev/ser1` is connected to whichever process registered that name; nothing in the client indicates that a driver, rather than a file, is on the other end.

8.2 How a driver reaches its hardware

Three things a driver needs are not ordinary POSIX, and each is a request to taskman rather than a kernel privilege.

Registers. Device registers are mapped by an mmap-family call naming physical addresses. taskman owns the address space (§4.6) and decides whether to grant the mapping; the driver receives a pointer. No driver holds a capability to all of memory, and on QSOE/L the mapping is a capability to those frames and nothing else.

Contiguous, uncached memory. A descriptor ring that hardware reads must be physically contiguous, and it must not be cached, which on RISC-V means an explicit page-table attribute. Drivers obtain such memory from taskman as a distinct kind of allocation rather than by hoping an ordinary one is suitable. This is a place where a correct-looking system fails intermittently: a descriptor smaller than a cache block, sharing that block with another, is written back as a unit and one field silently overwrites the other. Structures that hardware and software both touch are allocated uncached, not maintained by hand.

Interrupts. A driver attaches an *interrupt service thread* (IST) to a source. The kernel then schedules that thread when the interrupt fires — and because Skimmer schedules by strict priority (§3.1), an IST displaces lower-priority work on its hart immediately. There is no interrupt *handler* in the monolithic sense running in an alien context with a restricted vocabulary: an IST is an ordinary thread that happens to be woken by hardware, and it may take locks, send messages and call the library like any other.

Note what is absent from all three. There is no driver API, no exported kernel symbol table, no module format, and no set of functions available to drivers and to nobody else. A driver's whole vocabulary is the one the shell has.

8.3 The console has two directions

A console is the clearest case of two drivers having to compose without either knowing about the other, and QSOE's answer is worth the space because it generalizes.

On a machine with a screen and a keyboard, the two do not arrive together. The screen is usable the moment the firmware hands its controller over (§8.5); the keyboard waits on a USB stack that enumerates much later, if at all. Pulling input from a keyboard would make the console depend on a driver that may never exist.

So input is **pushed**. The console server publishes its path at boot and parks any reader that arrives. A keyboard driver, whenever it comes up, opens that same path and injects decoded characters through the console's `control` method. Neither server needs to know whether the other exists, there is no discovery loop, and no ordering requirement between them.

What is injected is *characters*, not scancodes. Translating from whatever the hardware speaks — HID usage codes, a PS/2 set, a serial protocol — belongs in the driver that understands that hardware, not in every console that might display the result.

A read with nothing to give defers. It does not report end-of-file, and the distinction is the whole reason a screen becomes a terminal before any keyboard exists. End-of-file is a lie about capability: it tells a reader "this device will never speak," and a `getty` that believes it spins through banner after banner forever. Deferring says "nothing yet," which is true, and a `getty` that hears it prints its prompt and waits — which is exactly what a terminal with nobody typing at it should look like. When a keyboard finally injects a character, the same parked read completes. No code on that path changed when the keyboard arrived, which is the test of whether the shape was right.

Deferral here is the framework's ordinary deferred reply (§6.4); the console is not doing anything a driver author cannot do.

8.4 Where synchronous IPC stops

Send/Receive/Reply is right for control and wrong for bulk transport, and a network interface is bulk transport wearing a `read`/`write` costume. Where the boundary falls is an architectural decision, so it belongs here rather than in a driver's own documentation.

Carrying one frame per message means every frame becomes a synchronous rendezvous: a parked reader, a deferred reply and two copies, repeated as often as frames arrive. It also caps the payload at the IPC message size, above which a write must either fail or — worse, and this is what happened — split a frame into two useless halves, put both on the wire, and report success.

QSOE's network drivers therefore sit behind a common interface, `libnetdev`, whose fast path carries no messages at all. The driver owns a region of page-sized frame buffers, hands their physical addresses to the controller, and the client maps the very same pages: hardware writes a frame once and both processes read it where it landed. What crosses between them

is a *pulse* — asynchronous, no reply — saying "look at the ring." `MsgSend` survives only for `control`, which is what it is good at.

Three controllers as different as a Cadence GEM, a DesignWare EQoS and a virtio device sit behind one such ring, which is the evidence that the abstraction is at the right level. The design notes acknowledge FreeBSD's netmap as prior art consulted rather than copied: a slot names its buffer instead of implying it from the slot number, so a controller may complete receives out of order with nobody copying to put them back in order; and the consumer releases slots that the producer refills, rather than the two sharing a count.

The older per-message path was *deleted* rather than kept alongside as a fallback. Two ways to move a frame means one of them rots, and a driver that cannot offer a ring cannot carry frames at all — which is a better failure than quietly moving less traffic by another route.

8.5 Firmware hand-off

QSOE has no display driver, yet it puts a login prompt on a monitor. It does so by accepting a device the firmware already initialized.

The mechanism is a **Controller Handoff Block** (CHB): the firmware brings a video controller up into a text mode, leaves a character grid scanning out, and publishes a description of it — where the cell array lives, how a cell is encoded, how to request a repaint. A consumer that finds the block knows a text-mode controller is up and how to drive it. The console server (`devc-hficon`) writes text into that grid.

Two aspects are architecture rather than convenience. The controller's silicon does not describe any of this: the cell array and the repaint doorbell are software constructs, so the block is the *only* authoritative source, and its presence is the signature that the arrangement holds at all. And on a controller whose character generator is itself a processor, the block must say which processors the firmware still owns — an operating system that onlines them all would stop the thing painting its own console.

The published block is deliberately not QSOE-specific. It is an ABI between a firmware module and whatever boots next, and QSOE is one consumer of it.

The same driver, two kernels, and what it took. The console runs on both variants, and the shared `devc-hficon` binary is the whole of the userspace story. Getting there on QSOE/L was entirely a question of *reaching* the controller, which is worth recording because it is the kind of constraint a capability kernel imposes and a conventional one does not.

Three things had to be true at once. The firmware's handoff block sits in ordinary RAM, which on seL4 belongs to the capability system, so it must be carved out of the untypeds handed to the root task before anything can map it — a device-tree reservation does that. And an seL4 untyped hands out frames from an index that only ever *advances*, so two consumers sharing one untyped must map in ascending physical order or the lower one is refused for the rest of the boot. That bit twice: once between the console and the disk, whose registers sit in one aperture, and again inside the console driver, which was asking for its own regions in the order the code wanted them rather than the order the allocator could serve.

None of it is visible above the seam. The driver now sorts its regions by address before mapping them, and taskman keeps a bounded window below each request so a driver that starts later is not stranded by one that started first. The lesson generalizes past this device: on a capability kernel the *order* in which drivers claim memory is part of the system's design, not an accident of an init script.

8.6 Finding the hardware

Drivers are FDT-driven, like the kernels beneath them (§10.1): a driver reads the flattened device tree to find its controller's registers and interrupt, rather than being compiled for a board. This is why a board is a link-time property of the kernel alone (§2.5) and not a property of userspace at all — the same driver binaries serve every board.

Devices on PCI are enumerated instead, and QSOE uses **MSI and MSI-X only**; legacy INTx is a deliberate non-goal. Message-signalled interrupts are what the modern controllers QSOE targets provide, and the shared-line arbitration that INTx requires buys nothing on any of them.

8.7 What a driver must never do

The discipline of §6.5 applies with particular force to drivers, because they are the programs most tempted to break it: a driver never calls seL4 or Skimmer directly, only POSIX and the QSOE IPC primitives. That is what lets one driver binary run on both variants, and it is checked by the simplest possible test — the drivers live in the shared userspace tree, where the per-kernel headers are not reachable.

Two further rules earn their place from experience rather than principle. A driver does not program clocks, resets or power gates that the firmware already brought up and is still using: two things programming one clock gate is worse than one thing doing it, and the one that already works is the firmware. And a driver on a hot path does not take the console lock per event; logging that serializes every interrupt turns a performance question into a deadlock question.

Chapter 9

The C Library and Dynamic Linking

The C library is where the whole system meets the application. Every chapter so far has described something an application reaches *through* libc: the IPC primitives (Chapter 5), the path-based `open` that finds a resource server (Chapter 6), the requests to taskman (Chapter 4). It is also where the last per-kernel difference hides — in a thin seam beneath an otherwise shared library.

9.1 One library, a thin seam

The C library is mostly shared. Its OS-independent body — the standard C functions, stdio, the memory allocator, the string and math routines, and the QNX-style API surface that presents Channels, Connections, Messages, and Pulses — lives once in the `libc` tree and is compiled for both variants. Around 85% of the library is shared at the source level.

Beneath it sits the per-kernel *seam* (`nq/libc` and `lq/libc`): the small layer where the library actually meets the kernel. This is where a `MsgSend` turns into the right trap or capability invocation, where a thread is created, where a pulse is delivered, and where the few per-variant data shapes are fixed. The seam is deliberately small — it is the third and last of the three things that differ between the variants (§2.2).

The library tree also carries the C runtime startup (`crt0`), so that everything a program needs in order to start lives in one place.

9.2 Why dynamic linking is mandatory

Chapter 2 stated that user programs are dynamically linked by requirement; here is why, precisely. Because the seam differs per kernel, the machine code that reaches the kernel is *not the same* on QSOE/N and QSOE/L. A statically linked binary would have to bake one kernel's seam into itself, and would then run on only that variant.

So a QSOE program contains no libc of its own. It is one position-independent ELF that, at load time, binds the *system's* `libc.so` — the one built for whichever kernel it is running on. The same binary therefore runs on either variant: the system supplies the correct library. This is the technical content of the "one userspace, two kernels" promise at the level of an individual executable, and it is why loading a program is work the system must do for it rather than an option a program may decline.

9.3 taskman is the loader

A dynamically linked program is conventionally started by an interpreter named in its `PT_INTERP` header: the kernel loads that interpreter, and the interpreter loads everything else. QSOE does

not work that way. There is no interpreter, and no QSOE binary carries a `PT_INTERP` header at all.

taskman does the loading itself. When it spawns a program it parses the executable, walks its `DT_NEEDED` list, loads `libc.so` and every other object the program names, applies every relocation, and jumps straight to the program's entry point. The scope a symbol is resolved against is the ordinary ELF one: the executable first, then each library in the order it was named.

Placing the loader in the central server rather than in every process is not merely a simplification. A loader running inside a process can only put an object where that process can reach it, and with no file-backed `mmap` in QSOE that means reading each library into private anonymous memory — so an in-process loader *structurally cannot* share a library between programs. taskman already brokers shared frames between processes, so it is the only place in the system from which sharing is even expressible.

One choice matters for QSOE's real-time ambitions: binding is *eager*. Rather than resolving each function on its first call (the usual lazy-PLT scheme), every symbol is resolved before the program runs. Deferring symbol resolution to first call would put unbounded work — and a possible fault — in the middle of a time-critical path; resolving eagerly keeps the running program's timing predictable, which a hard-real-time system cannot do without.

9.4 Two limits at the foundation

Two properties of the loader as it stands are worth stating plainly, because they are not defects in anything above them — they are the floor everything above them rests on, and both will be felt long before anything else in the system becomes the limit.

9.4.1 Library text is not yet shared

taskman loads each object into its own address space, relocates it there, and then *moves* the pages into the new process; the slot it loaded through is left empty. Each process therefore holds a private copy of every library it uses, text included. For `libc.so` that is about 220 KB per process — roughly 200 KB of instructions and 22 KB of writable data — and those instruction bytes are identical in every process on the machine. A thousand programs is a fifth of a gigabyte of duplicated code; ten thousand is beyond what any board here carries.

The separation the sharing would need is already present. A library's text segment carries no relocations whatsoever — the code is position-independent and reaches its data through a global offset table — so once loaded it is byte-identical wherever it lands, while everything genuinely per-process (that offset table, `.data`, `.bss`) already lives in the writable segment beside it. Libraries are loaded at fixed addresses, and the mechanism that would map one copy into many processes is the same region broker a driver already uses to hand a shared frame to its clients. What is missing is the use of it, not the means.

9.4.2 Loading is serialized against everything else

taskman serves its channel from a single receive loop. While it is loading a program it is not answering anything else: not an `open`, not a path resolution, not an `mmap`, for any process on the machine. Spawning therefore does not merely serialize against other spawns — it serializes against the whole system's traffic through taskman.

The cost of one spawn is dominated by neither of the things one would expect. Relocation is cheap: a few hundred entries for `libc.so` and a few dozen for a small library. What is expensive is reading the executable, which arrives from a filesystem in messages of under a kilobyte apiece, so a program of a hundred kilobytes costs upward of a hundred round trips — each one a send and a reply on that single thread — followed by the allocation and copying of its pages.

The order in which these are worth addressing follows from that. Sharing library text removes both the duplicated memory and the copying. Reading an executable in bulk, through the large-message path the system already has, turns a hundred round trips into a handful. Only after both does making taskman's dispatch concurrent become the next limit — and it is much the most invasive of the three, because the rule that a forwarded-to server must not call back into taskman before replying, and the deferred-reply machinery built on it, both assume the single thread.

9.5 The split of state: library versus taskman

A QNX-style system divides its bookkeeping between the per-process C library and the central server, and QSOE follows that division. Some state is private to a process and lives in its library: the table of its open connections, its side of the coid namespace, its signal dispositions. Other state is system-wide and lives in taskman: the channel registry, the process table, the pathname space.

The guiding rule is to keep in the library whatever can correctly be kept there, so that a common operation does not require a message to taskman for every step. A `sigaction`, for instance, is a process-local update in the library — taskman never sees a signal disposition — while a `spawn`, which must allocate a new process system-wide, is necessarily a request to taskman. This is the same mechanism-versus-policy line drawn one level higher: the library holds the policy that is safely process-local; taskman holds the policy that must be central.

9.6 The QNX API is part of libc

In QNX the message-passing API is simply part of the C library, and QSOE arranges the same. The QNX-style surface — `ChannelCreate`, `ConnectAttach`, `MsgSend` and its family, the `Sync*` primitives, the timer and thread calls — is built into `libc.so` alongside the POSIX functions, not delivered as a separate archive a program must remember to link. A program links one library and has both the standard C interface and the QNX system interface.

Chapter 10

Boot

This final chapter follows each variant from power-on to `/sbin/init`. The two paths share a prologue and a destination but differ in the middle — and the way they differ says something about the two kernels' natures. The chapter closes with the bootloader and the self-booting disk image that are common to both.

10.1 The common prologue

Both variants begin the same way. The platform firmware — OpenSBI in M-mode — initializes the hardware and hands control to S-mode with a pointer to the flattened device tree (FDT) in a register. The FDT is the system's self-description: the harts and their capabilities, the memory map, the interrupt controller, the console and the timer. Both kernels are FDT-driven, reading it to discover the machine rather than being compiled for one board.

From here the two paths diverge in how the kernel and the first userland are packaged.

10.2 QSOE/N: kernel plus initrd

QSOE/N boots in the conventional kernel-plus-initrd shape. Two artifacts are involved:

- `skimmer.bin` — the Skimmer kernel as a *flat RISC-V Image*, not an ELF. The loader jumps straight to the image's entry, as it would for a Linux `Image`.
- `modpkg.cpio` — the shared userspace package, supplied separately as an initial RAM disk.

OpenSBI enters `skimmer.bin`; the kernel finds the initrd by reading its location from the `/chosen` node of the FDT, parses the cpio archive, and starts taskman from it, which goes on to `/sbin/init`. Under QEMU the initrd is handed over with `-initrd`; under the mr-bml bootloader it is loaded by a `modpkg` directive. Nothing is embedded — kernel and initrd remain two files, joined only at boot through the device tree.

10.3 QSOE/L: one self-contained image

QSOE/L boots from a single self-contained ELF, assembled by nesting each stage inside the previous one:

1. OpenSBI enters the **elfloader**, a small loader that knows how to unpack and start an seL4 system.
2. The elfloader image embeds an archive of the **seL4 kernel** and **taskman** (the root task), included directly into its data at build time.

3. taskman, in turn, embeds the userspace package, so that once it runs it can start `/sbin/init` with no external file.

The result is one ELF — the `qsoe-l-*.elf` of the downloads — that needs nothing alongside it to reach a shell. This nesting reflects seL4's nature: its initial set of resources and the root task are fixed at build time and bootstrapped through capabilities, so packaging the whole system as one statically assembled image is the natural fit.

10.4 Two strategies, one destination

The contrast is deliberate (Table 10.1). QSOE/N keeps the kernel and the initrd as separate artifacts discovered through the device tree — the familiar, flexible shape. QSOE/L folds everything into one ELF, matching the static, capability-bootstrapped way an seL4 system comes up. Both arrive at the same place: taskman running, and `/sbin/init` starting the shared userspace described in the rest of this manual.

	QSOE/N (Skimmer)	**QSOE/L (seL4)**
Kernel form	flat RISC-V Image (`skimmer.bin`)	ELF, via elfloader
Userspace	separate initrd (`modpkg.cpio`)	embedded in the image
Joined	at boot, via the FDT `/chosen`	at build, by nesting
Artifacts	two files	one self-contained ELF

Table 10.1: The two boot strategies.

10.5 The bootloader and the self-booting image

On real hardware, and in the published disk image, a bootloader sits before all of this. QSOE uses **mr-bml**, its own GRUB-derived UEFI bootloader: an EFI application that reads ext-formatted partitions, presents a menu, and loads either variant — `kernel` plus `modpkg` for QSOE/N, or the self-contained ELF for QSOE/L.

This is what makes the downloadable image self-booting. The disk carries mr-bml in its EFI System Partition and the kernels on an ordinary filesystem, so the firmware runs mr-bml, mr-bml shows the QSOE menu, and a chosen variant boots — with no external kernel argument, under QEMU exactly as on the Unmatched. The mechanics of building and running that image belong to the User Guide (`QSOE-USR`); here it is enough to see where the bootloader fits: in front of the two boot paths this chapter has described, feeding each the artifacts it expects.